App open ads

Page Summary

Select platform: Android (beta) NEW (/admob/android/next-gen/app-open) Android (/admob/android/app-open) iOS (/admob/ios/app-open) Unity (/admob/unity/app-open) Flutter (/admob/flutter/app-open)

This guide is intended for publishers integrating app open ads.

App open ads are a special ad format intended for publishers wishing to monetize their app load screens. App open ads can be closed by your users at any time. App open ads can be shown when users bring your app to the foreground.

Note: Specific format may vary by region.

App open ads automatically show a small branding area so users know they're in your app. Here is an example of what an app open ad looks like:

At a high level, here are the steps required to implement app open ads:

1. Create a manager class that loads an ad before you need to display it.
2. Show the add during app foregrounding events.
3. Handle presentation callbacks.

Prerequisites

- Set up Google Mobile Ads SDK (/admob/ios/quick-start).
- Know how to configure your device as a test device (/admob/ios/test-ads).

Always test with test ads

When building and testing your apps, make sure you use test ads rather than live, production ads. Failure to do so can lead to suspension of your account.

The easiest way to load test ads is to use our dedicated test ad unit ID for app open ads:

```
ca-app-pub-3940256099942544/5575463023
```

It's been specially configured to return test ads for every request, and you're free to use it in your own apps while coding, testing, and debugging. Just make sure you replace it with your own ad unit ID before publishing your app.

For more information about how Google Mobile Ads SDK test ads work, see Test Ads (/admob/ios/test-ads).

Implement a manager class

Your ad should show quickly, so it's best to load your ad before you need to display it. That way, you'll have an ad ready to go as soon as the user enters your app. Implement a manager class to make ad requests ahead of when you need to show the ad.

Create a new singleton class called `AppOpenAdManager`:

Swift Objective-C (#objective-c)
(#swift)

```
class AppOpenAdManager: NSObject {
  /// The app open ad.
  var appOpenAd: AppOpenAd?
  /// Maintains a reference to the delegate.
  weak var appOpenAdManagerDelegate: AppOpenAdManagerDelegate?
  /// Keeps track of if an app open ad is loading.
  var isLoadingAd = false
  /// Keeps track of if an app open ad is showing.
  var isShowingAd = false
  /// Keeps track of the time when an app open ad was loaded to discard expired ad.
  var loadTime: Date?
```

```
  /// For more interval details, see https://support.google.com/admob/answer/9341964
  let timeoutInterval: TimeInterval = 4 * 3_600

  static let shared = AppOpenAdManager()
```

)c8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L30-L44)

And implement its **`AppOpenAdManagerDelegate`** protocol:

SwiftObjective-C (#objective-c)
(#swift)

```
protocol AppOpenAdManagerDelegate: AnyObject {
  /// Method to be invoked when an app open ad life cycle is complete (i.e. dismissed o
  /// show).
  func appOpenAdManagerAdDidComplete(_ appOpenAdManager: AppOpenAdManager)
}
```

)c8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L21-L25)

Load an ad

The next step is to load an app open ad:

SwiftObjective-C (#objective-c)
(#swift)

```
func loadAd() async {
  // Do not load ad if there is an unused ad or one is already loading.
  if isLoadingAd || isAdAvailable() {
    return
  }
  isLoadingAd = true

  do {
    appOpenAd = try await AppOpenAd.load(
      with: "ca-app-pub-3940256099942544/5575463023", request: Request())
    appOpenAd?.fullScreenContentDelegate = self
    loadTime = Date()
  } catch {
    print("App open ad failed to load with error: \(error.localizedDescription)")
    appOpenAd = nil
    loadTime = nil
  }
  isLoadingAd = false
```

```
}
```
)c8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L63-L83)

Show an ad

The next step is to show an app open ad. If no ad is available, attempt to load a new ad.

SwiftObjective-C (#objective-c)
(#swift)

```
func showAdIfAvailable() {
  // If the app open ad is already showing, do not show the ad again.
  if isShowingAd {
    return print("App open ad is already showing.")
  }

  // If the app open ad is not available yet but is supposed to show, load
  // a new ad.
  if !isAdAvailable() {
    print("App open ad is not ready yet.")
    // The app open ad is considered to be complete in this example.
    appOpenAdManagerDelegate?.appOpenAdManagerAdDidComplete(self)
    // Load a new ad.
    return
  }

  if let appOpenAd {
    appOpenAd.present(from: nil)
    isShowingAd = true
  }
}
```
:8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L87-L114)

Show the ad during app foregrounding events

When the application becomes active, call **`showAdIfAvailable()`** to show an ad if one is available, or loads a new one.

Key Point: If you use Scenes (//developer.apple.com/documentation/uikit/app_and_environment/scenes), implement the **`sceneDidBecomeActive:`** (//developer.apple.com/documentation/uikit/uiscenedelegate/3197915-scenedidbecomeactive) method in your **`UISceneDelegate`** instead.

SwiftObjective-C (#objective-c)
(#swift)

```
func applicationDidBecomeActive(_ application: UIApplication) {
  // Show the app open ad when the app is foregrounded.
  AppOpenAdManager.shared.showAdIfAvailable()
}
```

0ba3abbc8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppDelegate.swift#L32-L35)

Handle presentation callbacks

To receive notifications for presentation events, you must assign the **`GADFullScreenContentDelegate`** to the fullScreenContentDelegate`s property of the returned ad:

SwiftObjective-C (#objective-c)
(#swift)

```
appOpenAd?.fullScreenContentDelegate = self
```

ɔc8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L74-L74)

In particular, you'll want to request the next app open ad once the first one finishes presenting. The following code shows how to implement the protocol in your **`AppOpenAdManager`** file:

SwiftObjective-C (#objective-c)
(#swift)

```
func adDidRecordImpression(_ ad: FullScreenPresentingAd) {
  print("App open ad recorded an impression.")
}

func adDidRecordClick(_ ad: FullScreenPresentingAd) {
  print("App open ad recorded a click.")
}

func adWillDismissFullScreenContent(_ ad: FullScreenPresentingAd) {
  print("App open ad will be dismissed.")
}

func adWillPresentFullScreenContent(_ ad: FullScreenPresentingAd) {
  print("App open ad will be presented.")
}
```

```
func adDidDismissFullScreenContent(_ ad: FullScreenPresentingAd) {
  print("App open ad was dismissed.")
  appOpenAd = nil
  isShowingAd = false
  appOpenAdManagerDelegate?.appOpenAdManagerAdDidComplete(self)
  Task {
    await loadAd()
  }
}

func ad(
  _ ad: FullScreenPresentingAd,
  didFailToPresentFullScreenContentWithError error: Error
) {
  print("App open ad failed to present with error: \(error.localizedDescription)")
  appOpenAd = nil
  isShowingAd = false
  appOpenAdManagerDelegate?.appOpenAdManagerAdDidComplete(self)
  Task {
    await loadAd()
  }
}
```

3332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L120-L157)

Consider ad expiration

Important: App open ads will time out after four hours. Ads rendered more than four hours after request time will no longer be valid and may not earn revenue.

To make sure you don't show an expired ad, you can add a method to the app delegate that checks the elapsed time since your ad reference loaded.

In your **`AppOpenAdManager`** add a **`Date`** property called **`loadTime`** and set the property when your ad loads. You can then add a method that returns **`true`** if less than a certain number of hours have passed since your ad loaded. Make sure you check the validity of your ad reference before you try to show the ad.

SwiftObjective-C (#objective-c)
(#swift)

```
private func wasLoadTimeLessThanNHoursAgo(timeoutInterval: TimeInterval) -> Bool {
  // Check if ad was loaded more than n hours ago.
  if let loadTime = loadTime {
    return Date().timeIntervalSince(loadTime) < timeoutInterval
  }
```

```
  return false
}

private func isAdAvailable() -> Bool {
  // Check if ad exists and can be shown.
  return appOpenAd != nil && wasLoadTimeLessThanNHoursAgo(timeoutInterval: timeoutInte
}
```

)c8332d70dc65574afd96a/Swift/admob/AppOpenExample/AppOpenExample/AppOpenAdManager.swift#L48-L59)

Cold starts and loading screens

The documentation assumes that you only show app open ads when users foreground your app when it is suspended in memory. "Cold starts" occur when your app is launched but was not previously suspended in memory.

An example of a cold start is when a user opens your app for the first time. With cold starts, you won't have a previously loaded app open ad that's ready to be shown right away. The delay between when you request an ad and receive an ad back can create a situation where users are able to briefly use your app before being surprised by an out of context ad. This should be avoided because it is a bad user experience.

The preferred way to use app open ads on cold starts is to use a loading screen to load your game or app assets, and to only show the ad from the loading screen. If your app has completed loading and has sent the user to the main content of your app, don't show the ad.

Key Point: In order to continue loading app assets while the app open ad is being displayed, you should always load assets in a background thread.

Best practices

Google built app open ads to help you monetize your app's loading screen, but it's important to keep best practices in mind so that your users enjoy using your app. Make sure to:

- Wait to show your first app open ad until after your users have used your app a few times.
- Show app open ads during times when your users would otherwise be waiting for your app to load.
- If you have a loading screen under the app open ad, and your loading screen completes loading before the ad is dismissed, you may want to dismiss your loading screen in the `adDidDismissFullScreenContent` method.

Complete example on GitHub

Swift (//github.com/googleads/googleads-mobile-ios-examples/tree/main/Swift/admob/AppOpenExample)

Objective-C (//github.com/googleads/googleads-mobile-ios-examples/tree/main/Objective-C/admob/AppOpenExample)

Next steps

Learn more about user privacy (/admob/ios/privacy).